



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ciencias

Sistemas Operativos

Desarrollo de un Monitor Básico del sistema en C

Proyecto Final

Presenta:

Rojo Peña Manuel Ianluck

Profesor:

Javier León Cotonieto

Ayudante laboratorio:

Adrián Martínez Manzo

Grupo: 7004, 2026-2

Fecha de entrega:

27 de mayo, 2026

Objetivo

El objetivo de este proyecto es implementar un **monitor de sistema** en lenguaje C enfocado en el monitoreo del procesador y la memoria. La herramienta debe exponer al menos dos métricas dinámicas que se actualicen en tiempo real, y puede presentarse como una aplicación de línea de comandos o con interfaz gráfica básica.

Este reporte documenta el desarrollo incremental de dicha herramienta, organizado por **implementaciones**, cada implementación representa una etapa funcional completa que puede extenderse a lo largo de más de una sesión de trabajo. El criterio que tomamos de avance no es el tiempo, sino la funcionalidad que se agrega en cada implementación.

El plan de implementaciones es el siguiente:

1. **Primer Implementación. Escaneo e impresión básica:** lectura de `/proc/meminfo` y `/proc/[pid]/` para obtener métricas de memoria y lista de procesos, salida por `printf`. Se establece también la infraestructura de compilación con CMake y el entorno reproducible con Docker.
2. **Segunda Implementación. Monitoreo de red (extensión futura):** incorporación de captura y análisis de paquetes de red al estilo de `iftop`, mostrando emisor, receptor, protocolo y volumen de tráfico.
3. **Tercer Implementación. Terminal interactiva con ncurses:** sustitución del `printf` por una interfaz de terminal estilizada que se actualiza en tiempo real. Se introducen dos hilos POSIX, uno recolector de datos y uno de dibujado, de modo que la interfaz nunca se congele independientemente de la carga del sistema.

Primer Implementación: Escaneo e impresión básica

El objetivo de esta primera implementación fue establecer la base del proyecto, definir las estructuras de datos que representan el estado del sistema, leer las métricas de memoria y la lista de procesos activos desde el sistema operativo, e imprimirlos en la terminal con `printf`. No hay interfaz especial, no hay actualizaciones en tiempo real y no hay hilos. Se trata de un *escaneo único*, el programa arranca, toma una fotografía del sistema y termina.

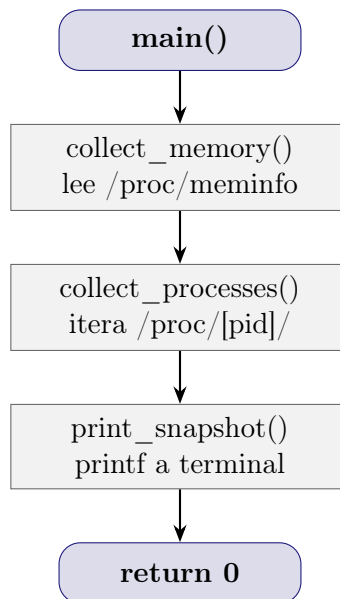
La razón: antes de introducir complejidad concurrente (hilos, mutex) o de presentación (ncurses, GTK), es necesario verificar que los datos que se leen son correctos. Con un `printf` simple nos permitimos confirmar que las métricas tienen sentido antes de construir encima de ellas.

Arquitectura del código

El proyecto se organiza en tres archivos fuente y un header:

- `include/monitor.h`: definición de las estructuras de datos y declaración de las funciones públicas.
- `src/monitor.c`: implementación de la recolección de datos y la impresión.
- `src/main.c`: punto de entrada; orquesta las tres llamadas en secuencia.

El flujo completo del programa es lineal:



Estructuras de datos: `monitor.h`

La primera decisión de diseño fue centralizar todos los datos del sistema en dos estructuras: `ProcessInfo` y `SystemSnapshot`.

`ProcessInfo` representa un único proceso, sus campos mapean directamente con los archivos que el kernel expone en `/proc/[pid]/`:

```

1 //Numero maximo de procesos que el snapshot puede almacenar
2 #define MAX_PROCS 512
3
4 /* Representa un unico proceso del sistema operativo,
5    cada campo viene de una fuente distinta dentro de /proc/[pid]/ */
6 typedef struct
7 {
8     int process_id;           //PID: identificador unico del proceso, fuente
9     int parent_process_id;    //PPID: PID del proceso que lo creo, fuente
10    char name[256];           //Nombre corto del ejecutable, max 15 chars
11    char cmdline[512];        //Comando completo con argumentos
12    char parent_name[256];     //Nombre del proceso padre
13    char state;               //Estado actual: R running, S sleeping, D waiting
14                               //I/O, Z zombie, T stopped
15    long vm_size_kb;          //Tamano total del espacio virtual en KB, incluye
16                               //todo lo reservado aunque no esteen RAM
17    long memory_kb;           //RSS en KB: paginas realmente en RAM ahora, es
18                               //la metrica de uso real de memoria
19 }
20
21 ProcessInfo;

```

Código 1: monitor.h struct ProcessInfo (líneas 12-21)

Cada campo tiene una fuente específica dentro de /proc:

- process_id viene del nombre del directorio en /proc;
- name de /proc/[pid]/comm; command_line de
- /proc/[pid]/cmdline; state, parent_process_id,
- memory_kb y vm_size_kb de /proc/[pid]/status.

SystemSnapshot agrupa las métricas globales de memoria y el arreglo de procesos en una sola estructura que se pasa por referencia entre funciones:

```

1 /* Fotografia completa del estado del sistema en un instante
2    agrupa metricas de memoria global y la lista de procesos */
3 typedef struct
4 {
5     long memory_total_kb;     //RAM total instalada
6     long memory_used_kb;      //RAM en uso real = total - free - buffers -
7                               //cached
8     long memory_free_kb;      //RAM completamente libre
9     long memory_buffers_kb;    //Cache de metadatos de filesystem
10    long memory_cached_kb;     //Cache de contenido de archivos
11    long swap_total_kb;        //Espacio de intercambio total
12    long swap_used_kb;         //Swap en uso = SwapTotal - SwapFree
13    ProcessInfo processes[MAX_PROCS]; //Arreglo estatico de procesos
14    encontrados
15    int process_count;         //Cuantas entradas validas hay en
16    processes[]
17 }

```

```
15 SystemSnapshot;
```

Código 2: monitor.h struct SystemSnapshot (líneas 23–34)

El arreglo `processes[MAX_PROCS]` es estático con `MAX_PROCS = 512`, de este modo evitamos asignación dinámica de memoria (`malloc`) en esta primera etapa, simplificando el código a costa de un límite fijo. En un sistema de escritorio típico el número de procesos rara vez supera los 400.

Restricción a Linux y portabilidad entre distribuciones

La herramienta depende de `/proc`, un sistema de archivos virtual exclusivo del kernel Linux, lo que significa que no compila ni corre en macOS, Windows ni BSD sin modificaciones.

Sin embargo, `/proc` está presente en todas las distribuciones basadas en Linux (Fedora, Ubuntu, Debian, Arch, Alpine) porque es una interfaz del kernel, no del sistema de paquetes, el mismo código que lee `/proc/meminfo` en Fedora funciona sin cambios en Ubuntu o Debian. El estándar que define su estructura está documentado en `man 5 proc` y ha sido estable durante décadas.

Para garantizar reproducibilidad entre máquinas con distintas distribuciones se introdujo un `Dockerfile`, descrito al final de este documento.

Lectura de memoria `collect_memory()`

La función `collect_memory()` lee el archivo `/proc/meminfo`, que el kernel regenera en cada acceso con los valores actuales de memoria. Su formato es una lista de campos con el patrón `Campo: valor kB`:

```
1  /* Funcion para leer /proc/meminfo y extrae las metricas de memoria del
   sistema /proc/meminfo
2     es generado por el kernel en cada lectura; cada linea tiene formato "
   Campo: valor kB" */
3 void
4 collect_memory(SystemSnapshot *system_snapshot)
5 {
6     FILE *f = fopen("/proc/meminfo", "r"); //abrimos el archivo virtual del
   kernel
7     if (!f)
8         return; //Si no se puede abrir, salir sin modificar el snapshot
9
10    char line[128]; //buffer para leer una linea a la vez
11    long value; //variable temporal donde sscanf deposita el numero
12
13    // Leemos linea por linea hasta EOF
14    while (fgets(line, sizeof(line), f)) {
15        /* sscanf intenta hacer match del patron en la linea
16         Devuelve el numero de campos que pudo leer (0 o 1)
17         solo guardamos el valor si el match fue exitoso (== 1) */
18        if (sscanf(line, "MemTotal: %ld kB", &value) == 1)
19            system_snapshot->memory_total_kb = value;
```

```

20     else if (sscanf(line, "MemFree: %ld kB", &value) == 1)
21         system_snapshot->memory_free_kb = value;
22     else if (sscanf(line, "Buffers: %ld kB", &value) == 1)
23         system_snapshot->memory_buffers_kb = value;
24     else if (sscanf(line, "Cached: %ld kB", &value) == 1)
25         system_snapshot->memory_cached_kb = value;
26     else if (sscanf(line, "SwapTotal: %ld kB", &value) == 1)
27         system_snapshot->swap_total_kb = value;
28     else if (sscanf(line, "SwapFree: %ld kB", &value) == 1)
29         system_snapshot->swap_used_kb = system_snapshot->swap_total_kb -
30         value;
31 }
32 fclose(f); //siempre cerrar el archivo para liberar el descriptor
33
34 /* Calculamos memoria realmente usada buffers y cached son reutilizables
35    por el kernel cuando hay presion de memoria, por eso no se cuentan
36    como "en uso real" por aplicaciones */
37 system_snapshot->memory_used_kb = system_snapshot->memory_total_kb
38                                 - system_snapshot->memory_free_kb
39                                 - system_snapshot->memory_buffers_kb
40                                 - system_snapshot->memory_cached_kb;
41 }

```

Código 3: monitor.c — collect_memory() (líneas 18–47)

El archivo `/proc/meminfo` tiene un formato fijo garantizado por el kernel, la cadena `"MemTotal: %ld kB"` es el patrón literal que aparece en el archivo, incluyendo los espacios y la unidad kB. `sscanf` devuelve el número de campos que pudo leer exitosamente; al verificar `== 1` nos aseguramos de que el match fue completo antes de guardar el valor, si la línea no coincide con el patrón, `sscanf` devuelve 0 y simplemente pasamos a la siguiente. Se cumple de manera análoga para los demás casos.

Linux usa la RAM libre para cachear contenido de archivos (`Cached`) y metadatos de filesystem (`Buffers`), esta memoria aparece como “usada” pero el kernel la libera inmediatamente cuando una aplicación la necesita. La fórmula correcta es:

$$\text{used} = \text{total} - \text{free} - \text{buffers} - \text{cached}$$

Esta es la misma fórmula que usan `htop` y `free -h`. Es por esta razón de qué `memory_used` no es simplemente `total - free`.

Lectura de procesos collect_processes()

```

1 /* Funcion para iterar /proc buscando directorios numericos (cada uno es
   un PID)
2    y extrae informacion de cada proceso vivo */
3 void
4 collect_processes(SystemSnapshot *system_snapshot)
5 {

```

```

6 DIR *d = opendir("/proc"); //abrimos el directorio /proc como un
  iterador
7 if (!d)
8     return;
9
10 struct dirent *entry; //entrada actual del directorio
11 system_snapshot->process_count = 0;
12
13 /* readdir devuelve la siguiente entrada cada vez que se llama,
14    NULL cuando no quedan mas entradas */
15 while ((entry = readdir(d)) != NULL && system_snapshot->process_count <
16     MAX_PROCS) {
17
18     /* Solo nos interesan entradas cuyo nombre empieza con digito
19        /proc tiene directorios como "net", "sys", "self" que ignoramos */
20     if (!isdigit(entry->d_name[0]))
21         continue;
22
23     int pid = atoi(entry->d_name);
24     ProcessInfo p;
25     memset(&p, 0, sizeof(p)); //inicializamos todos los campos a cero
26     p.process_id = pid;
27
28     char path[64]; //buffer para construir rutas como /proc/1234/comm
29     char line[256]; //buffer para leer lineas de los archivos de status
30     FILE *f;
31
32     /* Contiene el comando completo con argumentos separados por '\0'
33        ej: "/usr/bin/firefox\0--no-sandbox\0"
34        fread en lugar de fgets porque puede haber '\0' en medio */
35     snprintf(path, sizeof(path), "/proc/%d/cmdline", pid);
36     f = fopen(path, "r");
37     if (f) {
38         int len = fread(p.cmdline, 1, sizeof(p.cmdline) - 1, f);
39         fclose(f);
40         //Reemplazamos separadores '\0' por espacios para poder imprimirlo
41         for (int i = 0; i < len - 1; i++)
42             if (p.cmdline[i] == '\0') p.cmdline[i] = ' ';
43         p.cmdline[len] = '\0';
44     }
45
46     /* Nombre corto del ejecutable, truncado a 15 chars por el kernel
47        siempre disponible, incluso para hilos del kernel */
48     snprintf(path, sizeof(path), "/proc/%d/comm", pid);
49     f = fopen(path, "r");
50     if (!f)
51         continue; //si no podemos leer comm, el proceso ya termino saltar
52     fgets(p.name, sizeof(p.name), f);
53     p.name[strcspn(p.name, "\n")] = '\0'; //quitamos el newline final
54     fclose(f);
55
56     /* Archivo con multiples campos en formato "Campo:\tvalor"
57        leemos State, PPid, VmRSS y VmSize de aqui */
58     snprintf(path, sizeof(path), "/proc/%d/status", pid);

```

```

58     f = fopen(path, "r");
59     if (!f)
60         continue;
61     while (fgets(line, sizeof(line), f)) {
62         long val;
63         char c;
64         int iv;
65         if (sscanf(line, "State:\t%c", &c) == 1)
66             p.state = c;
67         else if (sscanf(line, "PPid:\t%d", &iv) == 1)
68             p.parent_process_id = iv;
69         else if (sscanf(line, "VmRSS:\t%ld kB", &val) == 1)
70             p.memory_kb = val;
71         else if (sscanf(line, "VmSize:\t%ld kB", &val) == 1)
72             p.vm_size_kb = val;
73     }
74     fclose(f);
75
76     /* Filtramos procesos del kernel: no tienen memoria de usuario (RSS =
77     0)
78     hilos como kworker, migration, rcu_sched no aportan informacion
79     util */
80     if (p.memory_kb == 0)
81         continue;
82
83     //Leemos el comm del proceso padre usando su PPID
84     if (p.parent_process_id > 0) {
85         snprintf(path, sizeof(path), "/proc/%d/comm", p.parent_process_id);
86         f = fopen(path, "r");
87         if (f) {
88             fgets(p.parent_name, sizeof(p.parent_name), f);
89             p.parent_name[strcspn(p.parent_name, "\n")] = '\0';
90             fclose(f);
91         }
92     }
93
94     //Guardamos el proceso en el arreglo y avanzamos el contador
95     system_snapshot->processes[system_snapshot->process_count] = p;
96     system_snapshot->process_count++;
97 }
98
99     closedir(d); //Liberamos el descriptor del directorio
100 }

```

Código 4: monitor.c collect_processes() (líneas 49-122)

En Linux cada proceso vivo tiene un directorio en `/proc` cuyo nombre es su PID, por ejemplo `/proc/1234/`. No existe una llamada de sistema que devuelva directamente la lista de PIDs activos; la forma estándar (la misma que usa `ps` y `top`) es iterar `/proc` y filtrar las entradas cuyo nombre sea numérico con `isdigit()`.

A diferencia de los demás archivos de `/proc`, `cmdline` no usa saltos de línea como separador entre argumentos usa el carácter nulo `'\0'`. Si usáramos `fgets`, la lectura se detendría en

el primer '\0' y perderíamos todos los argumentos, fread lee los bytes crudos y luego reemplazamos los '\0' internos por espacios para poder imprimirlos.

Los hilos internos del kernel (kworker, migration, rcu_sched) aparecen como entradas en /proc pero no tienen espacio de memoria de usuario asignado. Su VmRSS es 0, incluirlos no aporta información útil para un monitor de memoria y solo ocupan entradas del arreglo.

VmRSS (Resident Set Size) son las páginas del proceso que están físicamente en RAM en este momento. VmSize es todo el espacio de direcciones virtuales reservado, que puede incluir regiones que el kernel nunca ha cargado a RAM o que ya fueron movidas a swap. Para saber quién está *realmente* usando memoria, RSS es la métrica correcta.

Impresión y ordenamiento print_snapshot()

```
1  /* Funcion para imprimir el snapshot completo, primero metricas de memoria
2     luego tabla de procesos ordenada de mayor a menor RSS */
3  void
4  print_snapshot(const SystemSnapshot *system_snapshot)
5  {
6     /* --- seccion de memoria --- */
7     printf("\n===== Memoria =====\n");
8     printf("  Total:    %10ld KB  (%ld MB)\n",
9         system_snapshot->memory_total_kb, system_snapshot->memory_total_kb /
10        1024);
11     printf("  Usada:    %10ld KB  (%ld MB)\n",
12         system_snapshot->memory_used_kb, system_snapshot->memory_used_kb /
13        1024);
14     printf("  Libre:    %10ld KB  (%ld MB)\n",
15         system_snapshot->memory_free_kb, system_snapshot->memory_free_kb /
16        1024);
17     printf("  Buffers: %10ld KB  (%ld MB)\n",
18         system_snapshot->memory_buffers_kb, system_snapshot->
19         memory_buffers_kb / 1024);
20     printf("  Cache:   %10ld KB  (%ld MB)\n",
21         system_snapshot->memory_cached_kb, system_snapshot->memory_cached_kb
22         / 1024);
23     printf("  Swap:    %10ld KB usados / %ld KB total\n",
24         system_snapshot->swap_used_kb, system_snapshot->swap_total_kb);
25
26     /* --- procesos ordenados por RSS --- */
27     //Usamos memcpy para no modificar el arreglo original del snapshot;
28     //qsort ordena en sitio usando cmp_mem como criterio
29     ProcessInfo sorted[MAX_PROCS];
30     memcpy(sorted, system_snapshot->processes, system_snapshot->
31         process_count * sizeof(ProcessInfo));
32     qsort(sorted, system_snapshot->process_count, sizeof(ProcessInfo),
33         cmp_mem);
34
35     printf("\n===== Procesos con memoria (%d)
36         =====\n",
37         system_snapshot->process_count);
```

```

30 printf("%-6s  %-6s  %-16s  %-10s  %-10s  %-8s  %-15s  %s\n",
31        "PID", "PPID", "Nombre", "RSS KB", "VmSize KB", "Estado", "Padre", "
    Comando");
32 printf("%-6s  %-6s  %-16s  %-10s  %-10s  %-8s  %-15s  %s\n",
33        "-----", "-----", "-----", "-----",
34        "-----", "-----", "-----", "
    -----");
35
36 for (int i = 0; i < system_snapshot->process_count; i++) {
37     const ProcessInfo *p = &sorted[i];
38
39     //Truncamos cmdline a 40 chars, si esta vacio usar name como fallback
40     char cmd_trunc[41];
41     snprintf(cmd_trunc, sizeof(cmd_trunc), "%.40s",
42              p->cmdline[0] ? p->cmdline : p->name);
43
44     printf("%-6d  %-6d  %-16s  %-10ld  %-10ld  %-8s  %-15s  %s\n",
45            p->process_id,
46            p->parent_process_id,
47            p->name,
48            p->memory_kb,
49            p->vm_size_kb,
50            state_str(p->state),
51            p->parent_name,
52            cmd_trunc);
53 }
54
55 //Resumen final
56 if (system_snapshot->process_count > 0) {
57     printf("\n Mayor consumidor: %s (PID %d) %ld KB\n",
58            sorted[0].name, sorted[0].process_id, sorted[0].memory_kb);
59     printf(" Menor consumidor: %s (PID %d) %ld KB\n",
60            sorted[system_snapshot->process_count-1].name,
61            sorted[system_snapshot->process_count-1].process_id,
62            sorted[system_snapshot->process_count-1].memory_kb);
63 }
64 }

```

Código 5: monitor.c print_snapshot() (líneas 124–180)

Antes de imprimir, la función copia el arreglo de procesos con `memcpy` y lo ordena con `qsort` usando `cmp_mem` como comparador, se ordena una *copia* para no alterar el orden original del snapshot, lo cual será importante cuando en implementaciones futuras otros hilos lean esos datos concurrentemente.

El comparador `cmp_mem` usa el idioma $(b > a) - (b < a)$ en lugar de una resta directa $b - a$. La razón es que `memory_kb` es de tipo `long` y la resta podría producir *overflow* si los valores son muy grandes, corrompiendo el ordenamiento silenciosamente.

Punto de entrada: main.c

```

1 #include "monitor.h"

```

```

2 #include <string.h>
3
4 /* Punto de entrada del programa */
5 int
6 main()
7 {
8     SystemSnapshot system_snapshot;
9     /* Inicializamos toda la estructura a cero antes de usarla, sin esto los
10        campos
11        tendrían basura de memoria valores aleatorios que podrían hacer
12        fallar los cálculos */
13     memset(&system_snapshot, 0, sizeof(system_snapshot));
14
15     //Paso 1: leemos métricas de memoria desde /proc/meminfo
16     collect_memory(&system_snapshot);
17     //Paso 2: iteramos /proc y llenar el arreglo de procesos
18     collect_processes(&system_snapshot);
19     //Paso 3: imprimimos todo lo recolectado
20     print_snapshot(&system_snapshot);
21
22     return 0;
23 }

```

Código 6: main.c completo (líneas 1–17)

`main()` inicializa el snapshot con `memset` a cero antes de cualquier lectura, sin esto, los campos del struct contendrían basura de la pila y los cálculos de memoria producirían valores incorrectos. La secuencia es estrictamente lineal: recolectar memoria, recolectar procesos, imprimir.

Infraestructura de compilación: CMake

El proyecto usa CMake con dos archivos `CMakeLists.txt`, uno raíz que define el proyecto y estándar de C, y uno dentro de `src/` que lista los archivos fuente y enlaza las bibliotecas. Esta separación nos permitirá agregar nuevos módulos en futuras implementaciones modificando solo el `CMakeLists.txt` de `src/`.

La compilación completa (sin uso de contenedores) se realiza con:

```
cmake -B build && cmake --build build && ./build/src/sysmon
```

Entorno reproducible: Dockerfile

Un reto práctico de cualquier proyecto en C es la gestión de dependencias del sistema, en esta implementación las dependencias son mínimas (solo GCC y CMake), pero en implementaciones futuras se agregarán **ncurses** para la terminal interactiva y **GTK-4 con libadwaita** para la interfaz gráfica. Instalar estas bibliotecas varía entre distribuciones, el paquete se llama `libgtk-4-dev` en Debian/Ubuntu pero `gtk4-devel` en Fedora por ejemplo.

Para eliminar esta fricción se introdujo un `Dockerfile` basado en `debian:bookworm-slim`, se eligió Debian porque es la base más neutral y ampliamente compatible, y sobre Fedora

porque los paquetes de GTK-4 están igualmente actualizados en los repositorios de Debian. La imagen instala desde ahora las bibliotecas que se usarán en implementaciones futuras (`libncurses-dev`, `libgtk-4-dev`, `libadwaita-1-dev`) , aunque en esta etapa el binario no las enlaza. Esto documenta las dependencias del proyecto completo y evita modificar el Dockerfile en cada implementación.

```
1 FROM debian:bookworm-slim
2
3 ENV DEBIAN_FRONTEND=noninteractive
4
5 RUN apt-get update && apt-get install -y \
6     gcc \
7     cmake \
8     libncurses-dev \
9     libgtk-4-dev \
10    libadwaita-1-dev \
11    pkg-config \
12    && rm -rf /var/lib/apt/lists/*
13
14 WORKDIR /app
15 COPY . .
16
17 RUN cmake -B build && cmake --build build
18
19 CMD ["/build/src/sysmon"]
```

Código 7: Dockerfile

Para ejecutar con acceso a los procesos reales del sistema huésped:

```
docker build -t sysmon .
docker run --rm --pid=host sysmon
```

La opción `-pid=host` comparte el espacio de nombres de procesos del sistema huésped con el contenedor. Sin ella, el contenedor tiene su propio `/proc` aislado y solo vería sus propios procesos internos.

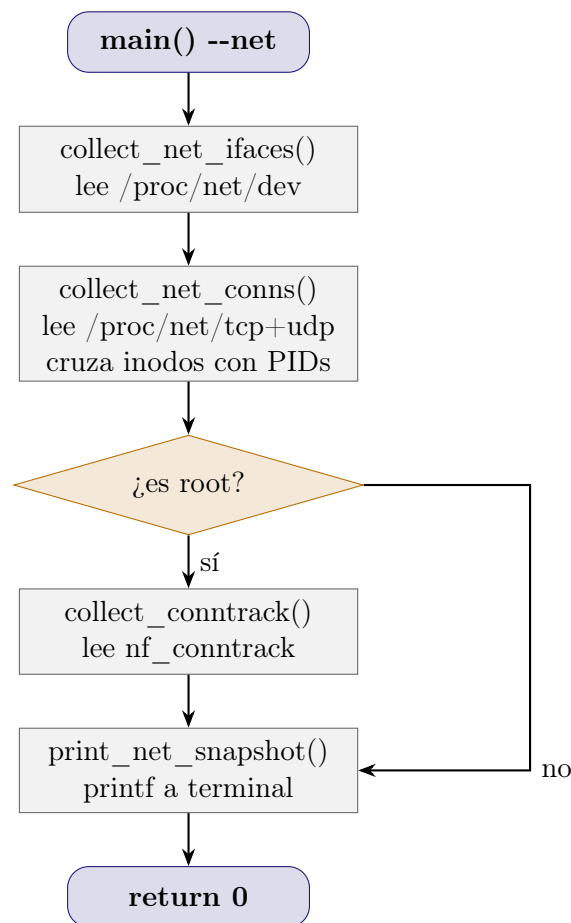
Segunda Implementación: Monitor de red

Descripción general

Esta implementación agrega un segundo modo de operación al programa, el monitoreo de la red. Al ejecutar `./sysmon --net`, el programa toma una fotografía del estado de la red y la imprime por terminal, de la misma forma que la primera implementación hacía con la memoria. No hay interfaz especial ni actualizaciones en tiempo real; el objetivo es, de nuevo, verificar que los datos leídos son correctos antes de integrarlos en la interfaz ncurses.

Se añaden dos archivos nuevos al proyecto: `include/net_monitor.h` y `src/net_monitor.c`. El archivo `src/main.c` recibe una bandera `--net` que decide qué modo ejecutar.

El flujo completo en modo red es el siguiente:



Cómo funciona la red en Linux

Antes de esta implementación, no tenía experiencia leyendo información de red desde el sistema operativo en C. En la implementación anterior, la fuente de datos simplemente: `/proc/meminfo` y los directorios de PID. Para la red, el primer desafío que se tuvo fue descubrir qué archivos leer y qué significan.

Linux expone el estado de la red a través de `/proc/net/`, un subdirectorio del mismo sistema de archivos virtual que ya conocíamos. Los archivos relevantes son:

- `/proc/net/dev`: estadísticas acumuladas de bytes y paquetes por interfaz de red desde el arranque del sistema.
- `/proc/net/tcp`: tabla de todas las conexiones TCP activas del sistema, con IPs y puertos en formato hexadecimal.
- `/proc/net/udp`: ídem para UDP.
- `/proc/net/nf_conntrack`: tabla del módulo netfilter con bytes y paquetes por conexión individual. Solo accesible con privilegios de root y requiere que el módulo tenga la contabilidad habilitada.

Lo que sí podemos leer **sin root**: `/proc/net/dev`, `/proc/net/tcp` y `/proc/net/udp`. Con ellos obtenemos interfaces, bytes totales por interfaz y la lista de conexiones activas con sus IPs. Lo que **requiere root**: `/proc/net/nf_conntrack` para ver cuántos bytes ha transferido cada conexión individual.

Estructuras de datos: `net_monitor.h`

Se definen tres estructuras que representan la red en un instante.

`IfaceStats` una interfaz de red

```
1 /* IfaceStats:
2    estadísticas de una interfaz de red /proc/net/dev */
3 typedef struct {
4     char name[32];    //nombre de la interfaz: eth0, wlan0, lo...
5     long rx_bytes;    //bytes recibidos desde que arranco el SO
6     long tx_bytes;    //bytes transmitidos desde que arranco el SO
7     long rx_packets;  //paquetes recibidos
8     long tx_packets;  //paquetes transmitidos
9     int is_virtual;   //1 si es loopback, bridge o virtual
10 } IfaceStats;
```

Código 8: `net_monitor.h`, struct `IfaceStats`

Cada campo tiene una fuente directa en `/proc/net/dev`:

- **name**: nombre de la interfaz, ej. `wlp1s0`, `lo`, `docker0`, es el identificador que el kernel asigna a cada dispositivo de red.
- **rx_bytes** / **tx_bytes**: total de bytes recibidos y transmitidos desde que arrancó el sistema, son contadores acumulados, no la velocidad actual. Para calcular velocidad se necesitan dos lecturas separadas por un intervalo de tiempo.
- **rx_packets** / **tx_packets**: número de paquetes procesados, un paquete es la unidad mínima de transmisión en red; su tamaño varía según el protocolo.

- `is_virtual`: campo que calculamos nosotros, no viene de `/proc`. Marca si la interfaz es virtual (no representa hardware real) para excluirla de los totales globales.

¿Qué es una interfaz virtual y por qué importa distinguirla? Una interfaz virtual es un dispositivo de red que existe solo en software, sin hardware físico detrás, los casos más comunes son:

- `lo` (loopback): permite que los procesos del mismo sistema se comuniquen entre sí por red sin salir al exterior. Todo lo que se envía a `127.0.0.1` pasa por aquí. Su tráfico no es tráfico real de internet.
- `docker0`, `br-*` (bridges de Docker): Docker crea interfaces de tipo *bridge* para conectar los contenedores entre sí y con el host. Si no hay contenedores corriendo, su tráfico es cero.
- `veth*`: interfaces virtuales que Docker y otras herramientas de contenedores crean en pares para conectar un contenedor al bridge.
- `virbr*`: bridges de *libvirt* para máquinas virtuales con KVM/QEMU.

Las interfaces físicas (`wlp1s0`, `eth0`, `enp3s0`) son las que tienen hardware real (tarjeta WiFi, Ethernet) y por las que fluye el tráfico real de internet. El campo `is_virtual` nos permite calcular totales que excluyan el ruido de las interfaces de software.

NetConn, una conexión activa

```

1 /* NetConn:
2    una conexion TCP o UDP activa asociada a un proceso
3    /proc/net/tcp + /proc/[pid]/fd/ para cruzar con PID */
4 typedef struct {
5     char protocol[8];           //"TCP" o "UDP"
6     char local_addr[48];        //direccion local ip:puerto en texto
7     char remote_addr[48];       //direccion remota ip:puerto en texto
8     char remote_host[128];      //hostname resuelto de la IP remota
9     char state[16];             //ESTABLISHED, LISTEN, TIME_WAIT, etc
10    int pid;                     //PID del proceso dueño del socket
11    char proc_name[256];         //nombre del proceso
12    long inode;                  //inodo del socket, sirve para cruzar datos
13    /* campos extra, solo disponibles con root via nf_conntrack */
14    long bytes_sent;             //bytes enviados en esta conexion
15    long bytes_recv;            //bytes recibidos en esta conexion
16    long packets_sent;
17    long packets_recv;
18    int has_traffic_data;        //1 si pudimos leer nf_conntrack
19 } NetConn;
```

Código 9: `net_monitor.h` struct `NetConn`

- `protocol`: TCP o UDP según el archivo de origen (`/proc/net/tcp` o `/proc/net/udp`).

- `local_addr` / `remote_addr`: dirección IP y puerto en formato texto "1.2.3.4:443", convertidos desde el hexadecimal que almacena el kernel.
- `remote_host`: resultado de la resolución DNS inversa de la IP remota. Si el servidor tiene un PTR record, aquí aparece su nombre de dominio en lugar de la IP numérica.
- `state`: estado actual de la conexión según el protocolo TCP.
- `pid`: identificador del proceso dueño del socket, se obtiene cruzando el campo `inode` con los file descriptors de cada proceso en `/proc/[pid]/fd/`.
- `inode`: número de inodo del socket en el sistema de archivos virtual, es la clave para cruzar conexiones con procesos.
- `bytes_sent`, `bytes_recv`, `packets_sent`, `packets_recv`: estadísticas de tráfico individuales por conexión. Solo disponibles cuando se corre como root y el módulo `nf_conntrack` tiene la contabilidad habilitada.
- `has_traffic_data`: bandera que indica si los campos de bytes contienen datos reales o están en cero por falta de permisos.

NetSnapshot, la fotografía completa

```

1 /* NetSnapshot:
2    fotografia completa del estado de red en un instante */
3 typedef struct {
4     IfaceStats ifaces[MAX_IFACES];
5     int iface_count;
6     NetConn conns[MAX_CONNS];
7     int conn_count;
8     //Totales globales sumados de todas las interfaces (excepto lo)
9     long total_rx_bytes;
10    long total_tx_bytes;
11 } NetSnapshot;

```

Código 10: `net_monitor.h` struct `NetSnapshot`

Al igual que `SystemSnapshot` en la implementación anterior, agrupa todos los datos de red en una sola estructura que se pasa por referencia. Los campos `total_rx_bytes` y `total_tx_bytes` son la suma de los bytes de todas las interfaces físicas (excluyendo virtuales), lo que da una visión global del tráfico real del sistema.

Por qué se necesita sudo

La mayor parte de la información de red es pública: cualquier usuario puede leer `/proc/net/tcp` y `/proc/net/dev`. Sin embargo, hay dos cosas que requieren privilegios de root:

1. **Leer `/proc/[pid]/fd/` de otros procesos.** Para cruzar los inodos de los sockets con PIDs, necesitamos abrir el directorio de file descriptors de cada proceso, el kernel restringe este acceso: un proceso solo puede ver los fd de procesos del mismo usuario.

Como root, podemos ver los fd de todos los procesos y por eso el cruce PID → proceso es más completo.

2. **Leer** `/proc/net/nf_conntrack`. Este archivo es propiedad del grupo `root` y no tiene permisos de lectura para otros usuarios. Contiene los contadores de bytes por conexión del módulo netfilter.

La función `is_root()` verifica esto con una sola llamada al sistema:

```
1 /* Funcion que verifica si es root
2    getuid() == 0 significa proceso corriendo como root */
3 int
4 is_root(void)
5 {
6     return getuid() == 0;
7 }
```

Código 11: `net_monitor.c` — `is_root()` (líneas 119–125)

`getuid()` es una llamada al sistema que devuelve el UID (User ID) real del proceso, en todos los sistemas Unix/Linux, el UID 0 está reservado para root.

Funciones auxiliares internas

Antes de las funciones públicas, `net_monitor.c` define cuatro funciones privadas (marcadas `static`) que se encargan de conversiones y utilidades.

`hex_to_ip` de hexadecimal a IP legible

```
1 static void
2 hex_to_ip(const char *hex, char *out, size_t out_size)
3 {
4     unsigned int addr;
5     sscanf(hex, "%X", &addr); //leemos el hex como entero sin signo
6     struct in_addr in;
7     in.s_addr = addr;         //asignamos sin reordenar bytes
8     //inet_ntoa convierte struct in_addr a "A.B.C.D"
9     snprintf(out, out_size, "%s", inet_ntoa(in));
10 }
```

Código 12: `net_monitor.c` — `hex_to_ip()` (líneas 10–24)

El kernel almacena las IPs en `/proc/net/tcp` como enteros hexadecimales en formato *little-endian*, por ejemplo, `0101A8C0` representa la IP `192.168.1.1`, la función lee el hex como entero sin signo con `sscanf`, lo asigna directamente a `in_addr.s_addr` sin reordenar bytes (porque el kernel ya lo almacena en el orden correcto para `inet_ntoa`), y `inet_ntoa` hace la conversión final a texto.

hex_to_port de hexadecimal a puerto decimal

```
1 static int
2 hex_to_port(const char *hex)
3 {
4     int port;
5     sscanf(hex, "%X", &port);
6     return port;
7 }
```

Código 13: net_monitor.c hex_to_port()

Los puertos también aparecen en hex en `/proc/net/tcp`. Por ejemplo, 01BB es el puerto 443 (HTTPS). La conversión es directa con `sscanf %X`.

tcp_state_str número de estado a texto

```
1 static const char*
2 tcp_state_str(int st)
3 {
4     switch (st) {
5     case 1:
6         return "ESTABLISHED";
7     case 2:
8         return "SYN_SENT";
9     case 3:
10        return "SYN_RECV";
11    case 4:
12        return "FIN_WAIT1";
13    case 5:
14        return "FIN_WAIT2";
15    case 6:
16        return "TIME_WAIT";
17    case 7:
18        return "CLOSE";
19    case 8:
20        return "CLOSE_WAIT";
21    case 9:
22        return "LAST_ACK";
23    case 10:
24        return "LISTEN";
25    case 11:
26        return "CLOSING";
27    default:
28        return "UNKNOWN";
29    }
30 }
```

Código 14: net_monitor.c tcp_state_str()

El kernel almacena el estado de cada conexión TCP como un entero en `/proc/net/tcp`, esta función mapea ese entero a su nombre estándar según la especificación del protocolo TCP (RFC 793). Los estados más relevantes:

- **ESTABLISHED**: conexión activa, ambos lados intercambian datos.
- **LISTEN**: el proceso espera conexiones entrantes (un servidor).
- **SYN_SENT**: el cliente envió el primer paquete del *handshake* TCP pero aún no recibió respuesta. Si persiste, puede indicar problemas de red.
- **TIME_WAIT**: la conexión terminó pero el kernel la mantiene brevemente para absorber paquetes retrasados.
- **CLOSE_WAIT**: el otro extremo cerró la conexión, esperando que este lado también cierre.

resolve_hostname DNS inverso

```

1 static void
2 resolve_hostname(const char *ip_str, char *out, size_t out_size)
3 {
4     struct sockaddr_in sa;
5     memset(&sa, 0, sizeof(sa));
6     sa.sin_family = AF_INET;
7     if (inet_pton(AF_INET, ip_str, &sa.sin_addr) != 1) {
8         strncpy(out, ip_str, out_size - 1);
9         return;
10    }
11    char host[128];
12    int result = getnameinfo(
13        (struct sockaddr *)&sa, sizeof(sa),
14        host, sizeof(host),
15        NULL, 0,
16        NI_NOFQDN    //nombre corto sin dominio completo
17    );
18    if (result == 0 && strcmp(host, ip_str) != 0)
19        strncpy(out, host, out_size - 1);
20    else
21        strncpy(out, ip_str, out_size - 1); //fallback a la IP numerica
22    out[out_size - 1] = '\0';
23 }

```

Código 15: net_monitor.c resolve_hostname()

Dado que las IPs en texto no son muy legibles, intentamos resolver cada IP remota a su nombre de dominio mediante una consulta DNS inversa (PTR record). `getnameinfo()` hace la consulta; si el servidor tiene un PTR record configurado, devuelve su hostname. Si no lo tiene (muchos servidores de CDN y nubes no lo configuran), la función devuelve la IP original como fallback. El flag `NI_NOFQDN` acorta el nombre eliminando el dominio completo.

format_bytes bytes a formato legible

```

1 static void
2 format_bytes(long bytes, char *out, size_t out_size)
3 {
4     if (bytes >= 1024 * 1024)
5         snprintf(out, out_size, "%.1f MB", bytes / (1024.0 * 1024.0));
6     else if (bytes >= 1024)
7         snprintf(out, out_size, "%.1f KB", bytes / 1024.0);
8     else
9         snprintf(out, out_size, "%ld B", bytes);
10 }

```

Código 16: net_monitor.c format_bytes()

Convierte un número de bytes crudos a una cadena legible con unidades, los umbrales son 1024 bytes para KB y 1024×1024 para MB. Se usa solo para mostrar los datos de tráfico de nf_conntrack.

collect_net_ifaces lectura de interfaces

```

1 void
2 collect_net_ifaces(NetSnapshot *ns)
3 {
4     FILE *f = fopen("/proc/net/dev", "r");
5     if (!f) return;
6     char line[256];
7     ns->iface_count = 0;
8     ns->total_rx_bytes = 0;
9     ns->total_tx_bytes = 0;
10    //Saltar las dos primeras lineas de encabezado
11    fgets(line, sizeof(line), f);
12    fgets(line, sizeof(line), f);
13    while (fgets(line, sizeof(line), f) && ns->iface_count < MAX_IFACES) {
14        IfaceStats *iface = &ns->ifaces[ns->iface_count];
15        int matched = sscanf(line,
16            " %31[~:]: %ld %ld %*ld %*ld %*ld %*ld %*ld %*ld %ld %ld",
17            iface->name,
18            &iface->rx_bytes, &iface->rx_packets,
19            &iface->tx_bytes, &iface->tx_packets);
20        if (matched != 5) continue;
21        //Marcamos interfaces virtuales
22        iface->is_virtual = (
23            strcmp (iface->name, "lo") == 0 ||
24            strncmp(iface->name, "br-", 3) == 0 ||
25            strncmp(iface->name, "veth", 4) == 0 ||
26            strncmp(iface->name, "virbr", 5) == 0 ||
27            strcmp (iface->name, "docker0") == 0);
28        //Acumulamos totales solo de interfaces fisicas reales
29        if (!iface->is_virtual) {
30            ns->total_rx_bytes += iface->rx_bytes;
31            ns->total_tx_bytes += iface->tx_bytes;
32        }
33        ns->iface_count++;

```

```

34 }
35 fclose(f);
36 }

```

Código 17: net_monitor.c collect_net_ifaces()

El archivo `/proc/net/dev` tiene dos líneas de encabezado que se saltan con dos llamadas a `fgets`. Cada línea de datos tiene el formato:

```
eth0: rx_bytes rx_pkts rx_err rx_drop ... tx_bytes tx_pkts tx_err ...
```

El patrón de `sscanf` usa `%*ld` para ignorar los campos que no necesitamos (errores, drops, fifo, frame), leyendo solo los bytes y paquetes. El patrón `%31[^:]` lee el nombre de la interfaz hasta el carácter `:` que actúa como separador.

collect_net_conns conexiones y cruce con PIDs

Esta es la función más compleja de la implementación, opera en dos fases.

Fase 1: leer conexiones desde `/proc/net/tcp` y `/proc/net/udp`:

```

1 void
2 collect_net_conns(NetSnapshot *ns)
3 {
4     ns->conn_count = 0;
5
6     const char *files[] = { "/proc/net/tcp", "/proc/net/udp" };
7     const char *protocols[] = { "TCP", "UDP" };
8
9     for (int fi = 0; fi < 2 && ns->conn_count < MAX_CONNS; fi++) {
10         FILE *f = fopen(files[fi], "r");
11         if (!f)
12             continue;
13
14         char line[512];
15         fgets(line, sizeof(line), f); //saltamos encabezado
16
17         while (fgets(line, sizeof(line), f) && ns->conn_count < MAX_CONNS) {
18             char local_hex[12], remote_hex[12];
19             char local_port_hex[6], remote_port_hex[6];
20             int state_num;
21             long inode;
22             int matched = sscanf(line,
23                 "%d: %8[^:]:%4s %8[^:]:%4s %X %s %s %s %d %d %ld",
24                 local_hex, local_port_hex,
25                 remote_hex, remote_port_hex,
26                 &state_num, &inode);
27
28             if (matched != 6)
29                 continue;
30
31             NetConn *c = &ns->conns[ns->conn_count];
32             strncpy(c->protocol, protocols[fi], sizeof(c->protocol) - 1);

```

```

33     c->inode = inode;
34     c->pid = -1;
35
36     char ip[INET_ADDRSTRLEN];
37     hex_to_ip(local_hex, ip, sizeof(ip));
38     snprintf(c->local_addr, sizeof(c->local_addr),
39              "%s:%d", ip, hex_to_port(local_port_hex));
40
41     hex_to_ip(remote_hex, ip, sizeof(ip));
42     snprintf(c->remote_addr, sizeof(c->remote_addr),
43              "%s:%d", ip, hex_to_port(remote_port_hex));
44
45     strncpy(c->state, tcp_state_str(state_num), sizeof(c->state) - 1);
46
47     if (state_num == 1 &&
48         strcmp(remote_hex, "00000000", 8) != 0) {
49     char remote_ip[INET_ADDRSTRLEN];
50     hex_to_ip(remote_hex, remote_ip, sizeof(remote_ip));
51     resolve_hostname(remote_ip, c->remote_host, sizeof(c->remote_host));
52     } else {
53     strncpy(c->remote_host, c->remote_addr, sizeof(c->remote_host) - 1);
54     }
55
56     ns->conn_count++;
57 }
58 fclose(f);
59 }
60 /* Fase 2: cruzar inodos con PIDs */
61 }

```

Código 18: net_monitor.c collect_net_conns() fase 1

Cada línea de /proc/net/tcp tiene el formato:

```

sl local_address rem_address st tx_queue:rx_queue tr:tm uid timeout inode
0: 0101A8C0:1F90 0202A8C0:01BB 01 00000000:00000000 00:0 0 1000 1 12345

```

El patrón de sscanf extrae la IP y puerto local y remoto en hex, el estado como entero, y el inodo. Los campos intermedios (colas, timer, uid) se ignoran con %*s y %*d.

Fase 2: cruzar inodos con PIDs:

```

1  /* Cruzamos inodos con PIDs
2     cada proceso tiene sus file descriptors en /proc/[pid]/fd/
3     los sockets aparecen como symlinks con destino "socket:[inode]"
4     iteramos todos los PIDs y todos sus fd hasta encontrar coincidencia
5  */
6  DIR *proc_dir = opendir("/proc");
7  if (!proc_dir)
8      return;
9
10 struct dirent *proc_entry;
11 while ((proc_entry = readdir(proc_dir)) != NULL) {
12     if (!isdigit(proc_entry->d_name[0]))
13         continue;
14     pid_t pid = atoi(proc_entry->d_name);
15     if (pid < 1)
16         continue;
17     char fd_path[100];
18     snprintf(fd_path, sizeof(fd_path), "/proc/%d/fd/", pid);
19     DIR fd_dir = opendir(fd_path);
20     if (!fd_dir)
21         continue;
22     struct dirent *fd_entry;
23     while ((fd_entry = readdir(fd_dir)) != NULL) {
24         if (!fd_entry->d_name[0])
25             continue;
26         char link_path[100];
27         snprintf(link_path, sizeof(link_path), "%s%s", fd_path, fd_entry->d_name);
28         char target[100];
29         if (readlink(link_path, target, sizeof(target)) < 0)
30             continue;
31         if (strncmp(target, "socket:", 8) == 0) {
32             char hex_inode[16];
33             strncpy(hex_inode, target + 8, 16);
34             hex_inode[16] = '\0';
35             if (strcmp(hex_inode, local_hex) == 0 ||
36                 strcmp(hex_inode, remote_hex) == 0) {
37                 c->pid = pid;
38                 c->inode = atoi(target + 8);
39                 return c;
40             }
41         }
42     }
43     closedir(fd_dir);
44 }
45 closedir(proc_dir);
46 return NULL;

```

```

12     continue;
13
14     int pid = atoi(proc_entry->d_name);
15     char fd_path[64];
16     snprintf(fd_path, sizeof(fd_path), "/proc/%d/fd", pid);
17
18     DIR *fd_dir = opendir(fd_path);
19     if (!fd_dir)
20         continue;
21
22     struct dirent *fd_entry;
23     while ((fd_entry = readdir(fd_dir)) != NULL) {
24         if (fd_entry->d_name[0] == '.')
25             continue;
26
27         char link_path[128], link_target[128];
28         snprintf(link_path, sizeof(link_path),
29                 "/proc/%d/fd/%s", pid, fd_entry->d_name);
30
31         ssize_t len = readlink(link_path, link_target, sizeof(link_target) -
32                                1);
33         if (len < 0)
34             continue;
35         link_target[len] = '\0';
36
37         if (strncmp(link_target, "socket:", 8) != 0)
38             continue;
39
40         long sock_inode;
41         sscanf(link_target, "socket:[%ld]", &sock_inode);
42
43         //Buscamos este inodo en nuestro arreglo de conexiones
44         for (int i = 0; i < ns->conn_count; i++) {
45             if (ns->conns[i].inode == sock_inode && ns->conns[i].pid == -1) {
46                 ns->conns[i].pid = pid;
47                 //Leemos nombre del proceso
48                 char comm_path[64];
49                 snprintf(comm_path, sizeof(comm_path), "/proc/%d/comm", pid);
50                 FILE *cf = fopen(comm_path, "r");
51                 if (cf) {
52                     fgets(ns->conns[i].proc_name, sizeof(ns->conns[i].proc_name), cf);
53                     ns->conns[i].proc_name[strcspn(ns->conns[i].proc_name, "\n")] = '\0';
54                 }
55                 fclose(cf);
56             }
57         }
58         closedir(fd_dir);
59     }
60     closedir(proc_dir);

```

Código 19: net_monitor.c collect_net_conns() fase 2

El mecanismo de cruce funciona porque en Linux todo socket es un archivo, y cada proce-

so tiene un directorio `/proc/[pid]/fd/` donde sus file descriptors abiertos aparecen como enlaces simbólicos. Un socket aparece con el destino `"socket:[12345]"` donde 12345 es su inodo. Al leer ese enlace con `readlink()` y comparar el inodo con los que registramos en la fase 1, encontramos qué proceso es dueño de cada conexión.

Desafío encontrado: sin root, `opendir(/proc/[pid]/fd)` falla para procesos de otros usuarios, devolviendo NULL. El código lo maneja con `if (!fd_dir) continue`, simplemente salta ese proceso y continúa. Con root, se puede abrir el directorio de todos los procesos y el cruce es completo.

collect_conntrack tráfico por conexión (solo root)

```
1 void
2 collect_conntrack(NetSnapshot *ns)
3 {
4     FILE *f = fopen("/proc/net/nf_conntrack", "r");
5     if (!f) {
6         fprintf(stderr, "    [!] nf_conntrack no disponible\n");
7         return;
8     }
9
10    char line[1024];
11
12    while (fgets(line, sizeof(line), f)) {
13        if (strncmp(line, "ipv4", 4) != 0)
14            continue;
15
16        char src_orig[48] = {0};
17        char dst_orig[48] = {0};
18        int sport_orig = 0;
19        int dport_orig = 0;
20
21        char *p;
22        p = strstr(line, "src=");
23        if (!p)
24            continue;
25        sscanf(p + 4, "%47s", src_orig);
26
27        //extraemos IP destino con el mismo patron
28        p = strstr(line, "dst=");
29        if (!p)
30            continue;
31        sscanf(p + 4, "%47s", dst_orig);
32
33        //extraer puerto origen: +6 salta "sport="
34        p = strstr(line, "sport=");
35        if (!p)
36            continue;
37        sscanf(p + 6, "%d", &sport_orig);
38
39        //extraer puerto destino: +6 salta "dport="
40        p = strstr(line, "dport=");
```



```

41     if (!p)
42         continue;
43     sscanf(p + 6, "%d", &dport_orig);
44
45     long bytes_orig = 0, bytes_reply = 0;
46     long pkts_orig = 0, pkts_reply = 0;
47     int has_acct = 0;
48
49     char *cursor = line;
50     int occ = 0;
51     while ((cursor = strstr(cursor, "bytes=")) != NULL) {
52         long val;
53         sscanf(cursor + 6, "%ld", &val); //+6 salta "bytes="
54         if (occ == 0) {
55             bytes_orig = val;
56             has_acct = 1;
57         } else {
58             bytes_reply = val;
59         }
60         occ++;
61         cursor++;
62     }
63
64     cursor = line; occ = 0;
65     while ((cursor = strstr(cursor, "packets=")) != NULL) {
66         long val;
67         sscanf(cursor + 8, "%ld", &val); //+8 salta "packets="
68         if (occ == 0)
69             pkts_orig = val;
70         else
71             pkts_reply = val;
72         occ++;
73         cursor++;
74     }
75
76     char local_str[48], remote_str[48];
77     snprintf(local_str, sizeof(local_str), "%s:%d", src_orig, sport_orig);
78     ;
79     snprintf(remote_str, sizeof(remote_str), "%s:%d", dst_orig, dport_orig);
80
81     for (int i = 0; i < ns->conn_count; i++) {
82         NetConn *c = &ns->conns[i];
83         if (strcmp(c->local_addr, local_str) == 0 &&
84             strcmp(c->remote_addr, remote_str) == 0) {
85             c->bytes_sent = bytes_orig;
86             c->bytes_recv = bytes_reply;
87             c->packets_sent = pkts_orig;
88             c->packets_recv = pkts_reply;
89             c->has_traffic_data = has_acct; /* 0 si no hay acct */
90             break; /* cada conexion es unica, no seguir buscando */
91         }
92     }

```

```

93
94     fclose(f);
95 }

```

Código 20: net_monitor.c collect_contrack()

El archivo /proc/net/nf_contrack tiene un formato por línea como:

```

ipv4 2 tcp 6 86399 ESTABLISHED src=192.168.1.1 dst=1.2.3.4 sport=12345
      dport=443 packets=100 bytes=15000 src=1.2.3.4 dst=192.168.1.1
      sport=443 dport=12345 packets=200 bytes=120000 [ASSURED]

```

Los campos src, dst, sport y dport aparecen dos veces: la primera vez para la dirección original (lo que enviamos) y la segunda para el reply (lo que recibimos). La función usa strstr en lugar de sscanf posicional porque el orden de los campos puede variar entre versiones del kernel.

Desafío importante: los campos bytes= y packets= solo aparecen en la línea si el módulo nf_contrack fue cargado con la contabilidad habilitada (nf_contrack_acct=1), si no está habilitada, la línea no tiene esos campos y has_acct queda en 0, lo que significa que no se muestran datos de tráfico por conexión, lo cual es preferible a mostrar ceros engañosos. Además, los contadores solo empiezan desde el momento en que se habilita la contabilidad, no retroactivamente para conexiones ya establecidas.

Para habilitarlo:

```

sudo sysctl net.netfilter.nf_contrack_acct=1

```

cmp_rx comparador para ordenamiento

```

1 static int
2 cmp_rx(const void *a, const void *b)
3 {
4     const IfaceStats *ia = (const IfaceStats *)a;
5     const IfaceStats *ib = (const IfaceStats *)b;
6     return (ib->rx_bytes > ia->rx_bytes) - (ib->rx_bytes < ia->rx_bytes);
7 }

```

Código 21: net_monitor.c cmp_rx()

Ordena las interfaces de mayor a menor por bytes recibidos, usando el mismo patrón de comparación sin resta directa que cmp_mem en la primera implementación, para evitar overflow en valores grandes de tipo long.

print_net_snapshot impresión

```

1 void print_net_snapshot(const NetSnapshot *ns) {
2     /* imprime todas las interfaces con sus contadores */
3     for (int i = 0; i < ns->iface_count; i++) { /* ... */ }
4 }

```

```

5  /* seccion de interfaces virtuales colapsada */
6  for (int i = 0; i < ns->iface_count; i++)
7      if (!iface->is_virtual) continue;
8      /* ... */
9
10 /* mayor y menor RX entre interfaces fisicas */
11 qsort(sorted_ifaces, real_count, sizeof(IfaceStats), cmp_rx);
12
13 /* conexiones: solo las que tienen PID resuelto,
14    filtrando UDP sin destino y mostrando datos de
15    trafico si has_traffic_data == 1 */
16 for (int i = 0; i < ns->conn_count; i++) {
17     if (c->pid == -1) continue;
18     if (strcmp(c->protocol, "UDP") == 0 &&
19         strcmp(c->remote_addr, "0.0.0.0", 7) == 0) continue;
20     /* ... printf ... */
21     if (c->has_traffic_data) {
22         format_bytes(c->bytes_sent, sent, sizeof(sent));
23         format_bytes(c->bytes_recv, recv, sizeof(recv));
24         printf("  ~%-10s v%-10s  pkts ~%ld/v%ld\n", ...);
25     }
26 }
27 }

```

Código 22: net_monitor.c print_net_snapshot()503

La función filtra dos tipos de entradas antes de imprimir: conexiones sin PID resuelto (proceso ya terminó o sin permisos) y sockets UDP sin dirección remota (sockets de escucha multicast que no representan conexiones reales). La columna de tráfico solo aparece cuando `has_traffic_data` es 1, evitando mostrar ceros que podrían interpretarse erróneamente como ausencia de tráfico.

Limitación: datos de velocidad en tiempo real

Los valores de RX/TX bytes en `/proc/net/dev` son contadores acumulados desde el arranque del sistema, no velocidad actual, para calcular cuántos KB/s se están transfiriendo en este momento se necesitan dos lecturas separadas por un intervalo de tiempo y calcular la diferencia. Esto no es posible en un escaneo único y se implementará en la siguiente fase, donde el hilo recolector guardará el snapshot anterior y calculará el delta en cada tick.

De igual forma, incluso con `root` y `nf_conntrack_acct` habilitado, si ninguna aplicación está enviando o recibiendo paquetes en el momento del escaneo, los contadores de esa conexión mostrarán solo el total histórico desde que se estableció la conexión, no actividad reciente. No hay forma de distinguir una conexión *idle* de una activa sin hacer la comparación entre dos snapshots.

Decisión sobre Docker y siguiente implementación

Durante la planeación del proyecto se consideró el uso de un contenedor Docker para garantizar reproducibilidad entre distribuciones, particularmente pensando en la dependencia de

GTK-4 y **libadwaita** para la interfaz gráfica. Sin embargo, dado que la implementación de la interfaz gráfica con GTK-4 queda fuera del alcance de este reporte, la necesidad de Docker desaparece: las únicas dependencias del proyecto son **gcc**, **cmake** y **libncurses-dev**, todas disponibles en cualquier distribución Linux con un solo comando.

La **tercera y última implementación** sustituirá el **printf** por una interfaz de terminal interactiva construida con **ncurses**. Se introducirán dos hilos POSIX (uno recolector y uno de dibujado) para que la pantalla se actualice en tiempo real sin congelarse bajo carga alta.

Para compilar y ejecutar el estado actual del proyecto:

```
# instalar dependencias (Fedora)
sudo dnf install gcc cmake ncurses-devel

# instalar dependencias (Debian/Ubuntu)
sudo apt install gcc cmake libncurses-dev

# compilar
cmake -B build && cmake --build build

# monitor de memoria
./build/src/sysmon

# monitor de red (sin root: conexiones sin datos de trafico)
./build/src/sysmon --net

# monitor de red (con root: incluye bytes por conexion)
sudo ./build/src/sysmon --net
```